

Sanos y Salvos

Plataforma Inteligente para la Localización y Recuperación de Mascotas Perdidas

Institución: Duoc UC

Asignatura: Desarrollo Full Stack III

Caso: Sanos y Salvos – Plataforma Inteligente para la localización y recuperación de mascotas perdidas

Integrantes:

- Juan Vargas Castillo
- Matilda Vargas Canseco

Sección: 001D

Fecha: 17 de abril de 2026.

Índice.

Sanos y Salvos.....	1
Plataforma Inteligente para la Localización y Recuperación de Mascotas Perdidas.....	1
1. Contextualización del caso.....	3
2. Selección de Patrones de Arquitectura.....	4
2.1 Justificación de los patrones.....	4
2.2 Contribución de los Patrones a la Solución.....	6
3. Diagrama de microservicios:.....	7
4. Comunicación entre Microservicios.....	8
4.1 Tipo de Comunicación.....	8
4.2 Uso de OpenFeign.....	8
4.3 Rol del API Gateway.....	9
4.4 Eficiencia Técnica y Operativa.....	9
5. Diagrama de comunicación de microservicios:.....	10
6. Arquitectura Propuesta y Uso de Contenedores.....	11
6.1 Servicio de Usuarios.....	11
6.2 Servicio de Mascotas.....	11
6.3 Servicio de Geolocalización.....	11
6.4 Servicio de Coincidencias.....	12
6.5 API Gateway.....	12
6.6 Relación entre Microservicios.....	12
6.7 Implementación mediante Contenedores (Docker).....	13
7. Diagrama de docker:.....	14
8. Evaluación del Diseño.....	15
9. Seguridad, Privacidad y Sostenibilidad.....	16
10. Conclusión.....	18
11. Implementación de Testing Unitario (Capa Service).....	19
11.1 Objetivo del testing.....	19
11.2 Alcance del testing.....	19
11.3 Herramientas utilizadas.....	19
11.4 Ejemplo de caso de prueba.....	20
11.5 Resultados generales.....	20
12. Implementación de SonarQube.....	21
12.1 Objetivo.....	21
12.2 Arquitectura de SonarQube.....	21
12.3 Configuración implementada.....	22
12.4 Ejecución del análisis.....	23
12.5 Resultados generales en SonarQube.....	23
12.6 Observaciones de calidad.....	23

12.7 Evidencia visual.....	24
13. Conclusión Final de la Evaluación 3.....	27

1. Contextualización del caso

En los últimos años, el aumento en la tenencia de mascotas ha generado una mayor preocupación por su bienestar y seguridad. En este contexto, la pérdida de mascotas se ha convertido en un problema frecuente en diversas ciudades, afectando tanto a los dueños como a organizaciones que colaboran en su recuperación, tales como clínicas veterinarias, refugios de animales y municipalidades. Actualmente, la gestión de información relacionada con mascotas extraviadas se realiza de manera descentralizada, utilizando redes sociales, formularios web básicos y comunicación informal. Esta situación provoca una serie de dificultades, tales como la duplicidad de datos, información incompleta, falta de estandarización y baja eficiencia en la búsqueda de coincidencias entre mascotas perdidas y encontradas.

Además, la ausencia de una plataforma centralizada impide visualizar patrones geográficos relevantes, como zonas con mayor incidencia de extravíos, lo que limita la capacidad de toma de decisiones por parte de organizaciones y autoridades.

Frente a esta problemática, la empresa “Sanos y Salvos” propone el desarrollo de una solución tecnológica moderna basada en una arquitectura de microservicios, que permita centralizar la información, estructurar los datos de manera eficiente y automatizar la detección de coincidencias mediante el análisis de características físicas y geográficas.

La solución contempla la interacción de múltiples actores, incluyendo dueños de mascotas, ciudadanos, clínicas veterinarias, refugios y municipalidades, cada uno con distintos roles y necesidades dentro del sistema. Por ello, se requiere una plataforma escalable, flexible y segura, capaz de manejar múltiples solicitudes concurrentes y adaptarse a futuros requerimientos.

En este contexto, el presente informe propone una arquitectura de software basada en microservicios, junto con la selección de patrones de diseño y herramientas tecnológicas, con el objetivo de construir una solución eficiente, mantenible y alineada con los requerimientos del cliente.

2. Selección de Patrones de Arquitectura

Para el desarrollo de la plataforma “Sanos y Salvos”, se seleccionan patrones de arquitectura y diseño que permiten construir una solución escalable, modular y eficiente, alineada con los requerimientos del cliente.

Los patrones seleccionados son:

- Arquitectura de Microservicios
- API Gateway
- Repository Pattern
- Service Layer Pattern
- DTO (Data Transfer Object)
- Factory Method
- Circuit Breaker

2.1 Justificación de los patrones

La selección de estos patrones responde a la necesidad de gestionar múltiples reportes de manera eficiente, asegurar la escalabilidad del sistema y facilitar la integración entre los distintos actores involucrados.

Arquitectura de Microservicios:

Permite dividir el sistema en servicios independientes (usuarios, mascotas, geolocalización y coincidencias), facilitando la escalabilidad, el mantenimiento y el desarrollo paralelo. Es fundamental para procesar múltiples reportes provenientes de diversas fuentes, asegurando alta disponibilidad.

API Gateway:

Actúa como punto de entrada único, centralizando el acceso a los microservicios.

Permite gestionar el enrutamiento, aplicar autenticación mediante JWT y controlar el acceso, reduciendo la complejidad del sistema y mejorando la seguridad.

Por ejemplo, cuando un usuario inicia sesión, la solicitud pasa por el API Gateway, el cual valida el token JWT antes de permitir el acceso al servicio de usuarios, evitando accesos no autorizados.

Repository Pattern:

Abstrae el acceso a la base de datos, separando la lógica de negocio de la persistencia. Facilita el mantenimiento, mejora la organización del código y permite cambios en la base de datos sin afectar otras capas. Se implementa mediante Spring Data JPA.

Por ejemplo, el servicio de mascotas utiliza repositorios para guardar y consultar información sin interactuar directamente con la base de datos, permitiendo cambiar el motor de persistencia sin modificar la lógica del sistema.

Service Layer Pattern:

Centraliza la lógica de negocio, separándose de los controladores y del acceso a datos. Permite organizar el código, reutilizar lógica y mejorar la escalabilidad del sistema, especialmente en procesos como el registro y validación de información.

Por ejemplo, al registrar una mascota, el controlador delega la lógica al servicio correspondiente, donde se validan los datos y se procesa la información antes de almacenarla.

DTO (Data Transfer Object):

Permite transferir datos entre capas sin exponer directamente las entidades de la base de datos, mejorando la seguridad, reduciendo el acoplamiento y optimizando la comunicación con el frontend.

Por ejemplo, al enviar información de una mascota al frontend, se utilizan DTOs para incluir solo datos relevantes como nombre, tipo y ubicación, evitando exponer información interna del sistema.

Factory Method:

Facilita la creación de objetos de forma flexible, permitiendo manejar distintos tipos de mascotas o reportes según el contexto, sin acoplar la lógica a clases específicas.

Por ejemplo, el sistema puede crear distintos tipos de reportes (mascota perdida o encontrada) utilizando una fábrica, sin necesidad de instanciar directamente cada clase concreta.

Circuit Breaker:

Permite manejar fallos en la comunicación entre microservicios, evitando errores en cascada y mejorando la resiliencia del sistema. Se aplica en servicios críticos como coincidencias y mascotas para mantener la disponibilidad.

Por ejemplo, si el servicio de geolocalización no responde, el servicio de coincidencias puede continuar operando con información parcial, evitando que todo el sistema falle.

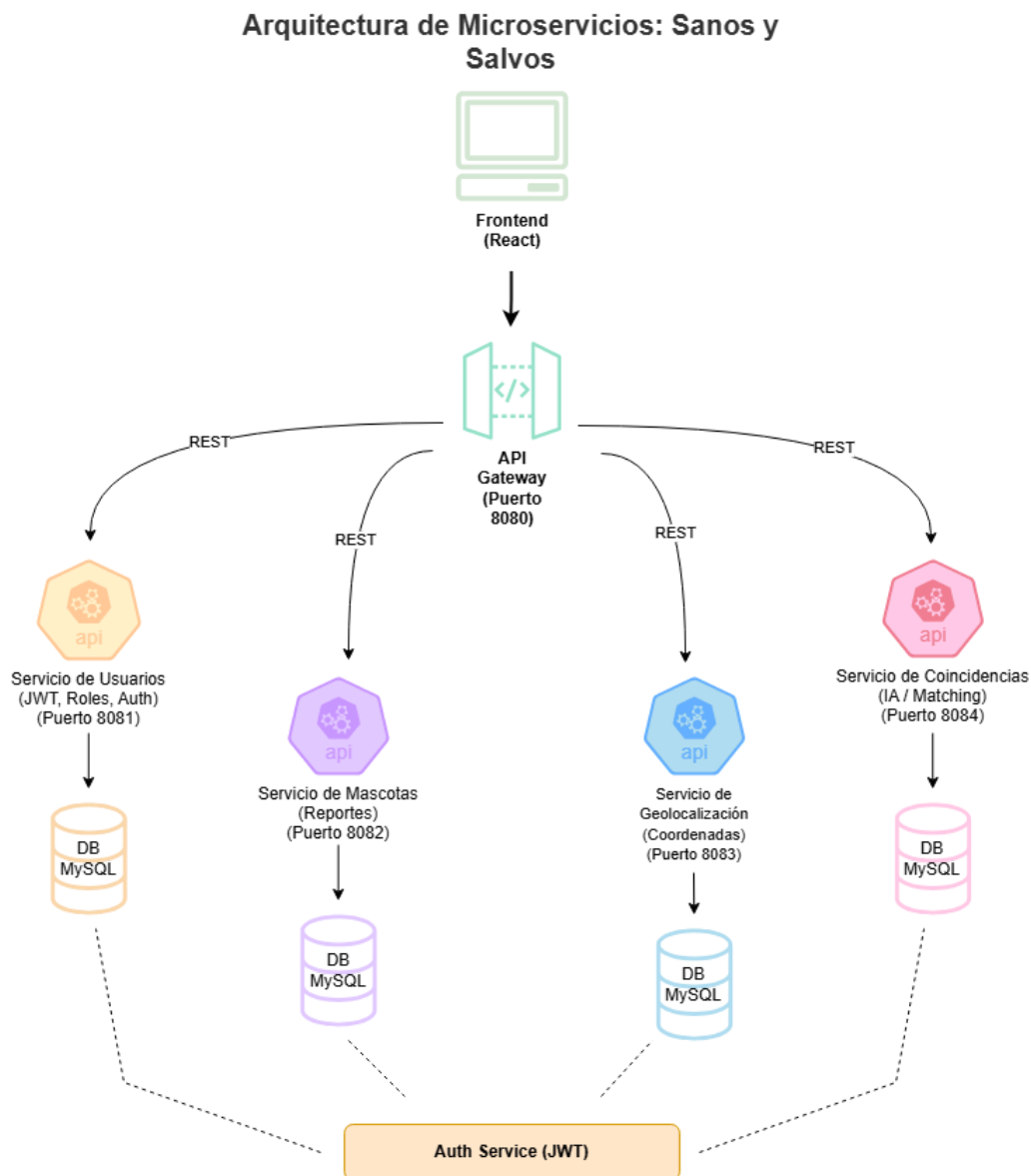
2.2 Contribución de los Patrones a la Solución

La combinación de estos patrones permite construir una arquitectura robusta y alineada con los requerimientos del cliente, ya que:

- Permite gestionar múltiples reportes de forma eficiente
- Facilita la integración entre usuarios, clínicas y refugios
- Mejora la escalabilidad del sistema
- Aumenta la resiliencia ante fallos
- Garantiza una estructura modular y mantenible

En conjunto, estos patrones permiten desarrollar una solución moderna, adaptable a futuras necesidades y orientada a mejorar la eficiencia en la localización de mascotas perdidas.

3. Diagrama de microservicios:



Este diagrama representa la arquitectura general del sistema basada en microservicios, donde el frontend se comunica con los distintos servicios a través de un API Gateway, el cual centraliza las solicitudes y aplica mecanismos de seguridad como la validación

de JWT. Cada microservicio (usuarios, mascotas, geolocalización y coincidencias) opera de forma independiente y se comunica mediante APIs REST, lo que permite reducir el acoplamiento y facilitar la escalabilidad. Además, cada servicio cuenta con su propia base de datos, asegurando el aislamiento de la información y permitiendo una mejor gestión de los datos. El uso de un servicio de autenticación basado en JWT refuerza la seguridad del sistema al controlar el acceso a los recursos.

4. Comunicación entre Microservicios

La comunicación entre los microservicios de la plataforma “Sanos y Salvos” se basa en servicios REST, utilizando el protocolo HTTP y el formato JSON para el intercambio de información. Este enfoque permite una integración estandarizada, interoperable y ampliamente utilizada en sistemas modernos.

4.1 Tipo de Comunicación

La arquitectura propuesta utiliza principalmente comunicación sincrónica, donde los microservicios interactúan entre sí a través de solicitudes HTTP en tiempo real. Este tipo de comunicación es adecuado para el sistema, ya que permite obtener respuestas inmediatas en operaciones críticas, como la consulta de mascotas o la generación de coincidencias.

4.2 Uso de OpenFeign

Para la comunicación interna entre microservicios se propone el uso de OpenFeign, herramienta que simplifica la creación de clientes HTTP en aplicaciones Spring Boot.

Su uso permite reducir la complejidad del código, facilitar la integración entre servicios y mejorar la mantenibilidad. En este sistema, será utilizado, por ejemplo, por el servicio de coincidencias para consumir información desde los servicios de mascotas y geolocalización.

4.3 Rol del API Gateway

El API Gateway actúa como intermediario entre el frontend y los microservicios, gestionando el enrutamiento de las solicitudes.

Su implementación permite centralizar el acceso, reducir la exposición directa de los servicios, mejorar la seguridad y simplificar la comunicación desde el cliente.

El flujo de comunicación es el siguiente:

Frontend (React) → API Gateway → Microservicios → Base de datos

4.4 Eficiencia Técnica y Operativa

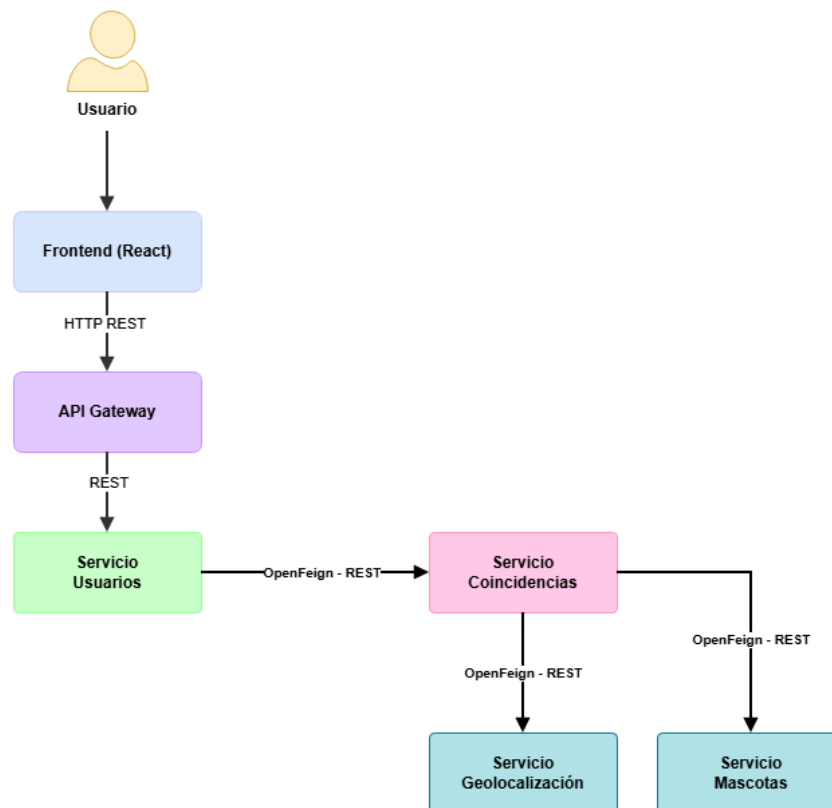
La combinación de REST, OpenFeign y API Gateway permite mejorar la eficiencia del sistema, ya que reduce el acoplamiento, facilita la escalabilidad, optimiza la integración entre servicios y mejora el mantenimiento del software.

Esto permite que la plataforma gestione múltiples solicitudes concurrentes, manteniendo un buen rendimiento y disponibilidad.

Además, el uso de OpenFeign reduce la duplicación de código en las llamadas entre microservicios, mientras que el API Gateway centraliza las solicitudes del cliente, disminuyendo la complejidad del frontend. Esto se traduce en menores tiempos de desarrollo, una integración más simple entre componentes y una reducción de errores en la comunicación entre servicios.

5. Diagrama de comunicación de microservicios:

Flujo de Comunicación entre Microservicios utilizando REST y OpenFeign



Este diagrama muestra el flujo de comunicación entre los distintos componentes del sistema, desde la interacción del usuario hasta la respuesta final. El frontend se comunica con el API Gateway mediante HTTP REST, el cual redirige las solicitudes al microservicio correspondiente.

En particular, se observa cómo el servicio de usuarios puede interactuar con el servicio de coincidencias utilizando OpenFeign, permitiendo realizar llamadas entre microservicios de forma simplificada y eficiente. A su vez, el servicio de coincidencias consume información desde los servicios de mascotas y geolocalización para analizar datos y generar resultados.

Este enfoque permite una comunicación estructurada en tiempo real, reduciendo la complejidad de integración y mejorando la mantenibilidad del sistema.

6. Arquitectura Propuesta y Uso de Contenedores

La solución para la plataforma “Sanos y Salvos” se basa en una arquitectura de microservicios, la cual divide el sistema en componentes independientes, cada uno encargado de una funcionalidad específica. Este enfoque permite mejorar la escalabilidad, mantenibilidad y disponibilidad, facilitando la gestión de múltiples reportes provenientes de distintos actores como dueños, ciudadanos, clínicas y refugios.

La arquitectura se compone de los siguientes microservicios:

6.1 Servicio de Usuarios

Gestiona el registro, autenticación y administración de usuarios, incluyendo distintos tipos de actores (dueños, ciudadanos, clínicas, refugios y municipalidades). Implementa autenticación mediante JWT y control de roles.

Se relaciona con el API Gateway para la validación de acceso y con el servicio de mascotas para la creación de reportes.

6.2 Servicio de Mascotas

Encargado de gestionar el registro de mascotas perdidas y encontradas, almacenando sus características principales y estado.

Se integra con los servicios de geolocalización y coincidencias para complementar la información y permitir su análisis.

6.3 Servicio de Geolocalización

Gestiona la información de ubicación de los reportes, permitiendo asociar coordenadas geográficas a cada mascota. Se integra con el servicio de mascotas y permite la visualización de información en mapas desde el frontend.

Para ello, se contempla la integración con la API de Google Maps, la cual permitirá representar geográficamente los reportes y mejorar la experiencia de usuario mediante mapas interactivos.

6.4 Servicio de Coincidencias

Analiza los reportes de mascotas perdidas y encontradas, comparando características como raza, color, tamaño y ubicación para identificar posibles coincidencias.

Consume información desde los servicios de mascotas y geolocalización.

6.5 API Gateway

Actúa como punto de entrada único al sistema, gestionando el enrutamiento de solicitudes, validación de JWT, control de acceso y manejo de errores.

Se comunica con todos los microservicios, permitiendo desacoplar el frontend de la lógica interna del sistema.

6.6 Relación entre Microservicios

Los microservicios se comunican mediante APIs REST, permitiendo el intercambio de información de forma estructurada.

En particular:

- El servicio de coincidencias consume datos de mascotas y geolocalización
- El servicio de mascotas se asocia a usuarios para registrar reportes
- El API Gateway gestiona todas las solicitudes del cliente

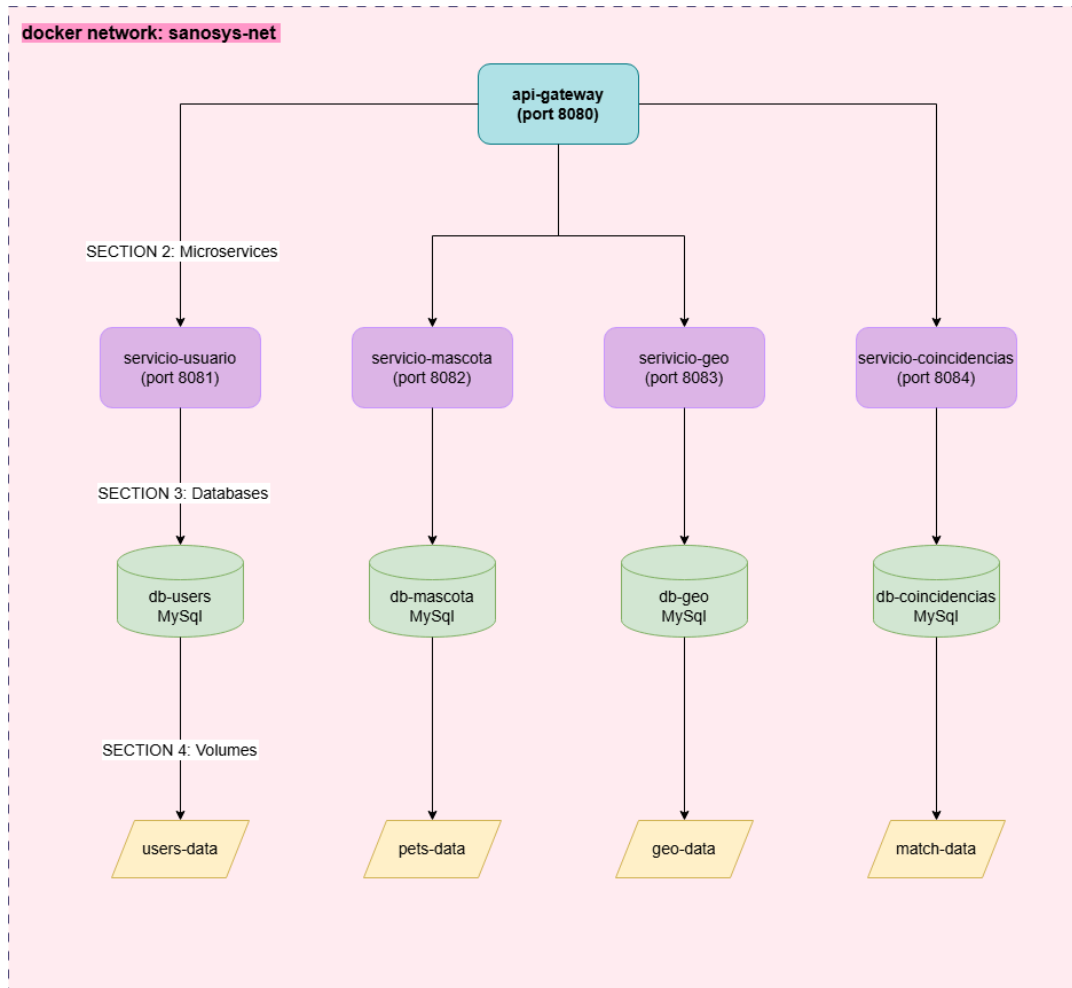
6.7 Implementación mediante Contenedores (Docker)

El sistema se implementa utilizando contenedores Docker, permitiendo empaquetar cada microservicio junto a sus dependencias en entornos aislados.

Cada servicio se ejecuta en un contenedor independiente, lo que permite aislar componentes, facilitar el despliegue, mejorar la escalabilidad y evitar problemas de compatibilidad.

Se utiliza Docker Compose para gestionar la ejecución conjunta de los servicios, incluyendo microservicios, API Gateway y bases de datos MySQL, permitiendo levantar el sistema de forma automatizada.

7. Diagrama de docker:



Este diagrama representa la implementación del sistema mediante contenedores Docker, donde todos los componentes se ejecutan dentro de un entorno aislado. El API Gateway y cada microservicio se despliegan como contenedores independientes, lo que permite una mayor portabilidad, consistencia y facilidad de despliegue. Cada microservicio se conecta a su propia base de datos, también contenida dentro del entorno, y estas a su vez utilizan volúmenes para asegurar la persistencia de los datos, incluso si los contenedores se detienen o eliminan.

Esta arquitectura basada en contenedores permite escalar servicios de manera independiente, optimizar el uso de recursos y facilitar la administración del sistema en distintos entornos.

8. Evaluación del Diseño

El diseño propuesto para la plataforma “Sanos y Salvos” responde de manera sólida a los requerimientos funcionales y técnicos del cliente, proporcionando una solución escalable, modular y eficiente. En primer lugar, la adopción de una arquitectura basada en microservicios permite gestionar múltiples reportes provenientes de distintas fuentes de manera concurrente, asegurando un adecuado manejo de la información y una alta disponibilidad del sistema. La separación de responsabilidades entre servicios facilita el desarrollo independiente, el mantenimiento y la evolución del sistema sin afectar su funcionamiento global.

La incorporación de un API Gateway permite centralizar el acceso a los servicios, mejorando la seguridad mediante control de autenticación y autorización, y simplificando la comunicación con el frontend. Esto reduce la complejidad del cliente y optimiza la experiencia de usuario. Desde el punto de vista técnico, el uso de herramientas y patrones como REST, OpenFeign y Circuit Breaker permite establecer una comunicación eficiente entre microservicios, reduciendo el acoplamiento y aumentando la resiliencia del sistema frente a fallos. En particular, el patrón Circuit Breaker evita la propagación de errores en cascada, lo que resulta fundamental en arquitecturas distribuidas.

A nivel operativo, la solución facilita la integración de múltiples actores, como usuarios, clínicas, refugios y municipalidades, permitiendo una colaboración efectiva y aumentando la probabilidad de recuperación de mascotas mediante el análisis automatizado de coincidencias. Asimismo, la implementación mediante contenedores Docker asegura la portabilidad del sistema, facilita su despliegue en distintos entornos y optimiza el uso de recursos, contribuyendo a la sostenibilidad y escalabilidad de la solución. No obstante, es importante considerar que la arquitectura de microservicios

introduce ciertos desafíos, como una mayor complejidad en la gestión de la comunicación entre servicios y posibles incrementos en la latencia debido a las múltiples interacciones HTTP. Sin embargo, estos desafíos se mitigan mediante el uso de patrones adecuados y una correcta organización de los servicios.

En términos generales, el diseño cumple con los principios de desacoplamiento, escalabilidad y mantenibilidad, alineándose con las necesidades del cliente y permitiendo la incorporación de futuras funcionalidades sin comprometer la estabilidad del sistema.

9. Seguridad, Privacidad y Sostenibilidad

La arquitectura propuesta incorpora de manera integral aspectos de seguridad, privacidad y sostenibilidad, con el objetivo de garantizar un sistema confiable, eficiente y alineado con buenas prácticas de la industria.

Seguridad:

El sistema implementa autenticación basada en JWT (JSON Web Tokens), permitiendo validar la identidad de los usuarios de forma segura en cada solicitud. Las contraseñas son protegidas mediante encriptación utilizando BCrypt, asegurando un almacenamiento robusto frente a posibles vulnerabilidades.

Además, se aplican mecanismos de validación de datos mediante Spring Validation, lo que permite prevenir errores y posibles ataques por entrada de datos inválidos.

El API Gateway cumple un rol clave como filtro de seguridad, centralizando el control de acceso, gestionando la autenticación y autorizando las solicitudes hacia los microservicios, evitando la exposición directa de estos.

Privacidad:

Se resguarda la información sensible de los usuarios mediante el uso de DTOs (Data Transfer Object), evitando la exposición directa de las entidades de base de datos.

El sistema implementa control de acceso basado en roles, asegurando que cada usuario solo pueda acceder a la información correspondiente a su perfil.

Asimismo, se limita el almacenamiento de datos únicamente a aquellos estrictamente necesarios para el funcionamiento del sistema, cumpliendo con principios de minimización de datos.

Sostenibilidad:

La arquitectura de microservicios permite una escalabilidad horizontal, posibilitando aumentar la capacidad del sistema según la demanda, sin afectar su funcionamiento global.

El bajo acoplamiento entre los componentes facilita la evolución del sistema, permitiendo incorporar mejoras o nuevas funcionalidades sin generar impactos significativos en otros servicios.

El uso de contenedores Docker optimiza la utilización de recursos y asegura una gestión eficiente de la infraestructura, facilitando además el despliegue y mantenimiento del sistema a largo plazo.

Asimismo, este enfoque permite reducir costos operativos, ya que solo se escalan los servicios que realmente lo requieren, evitando el uso innecesario de recursos computacionales.

10. Conclusión

El desarrollo de la plataforma “Sanos y Salvos” constituye una solución tecnológica integral frente a la problemática de mascotas extraviadas, permitiendo centralizar la información, mejorar la coordinación entre los distintos actores involucrados y optimizar los procesos de búsqueda y recuperación.

La adopción de una arquitectura basada en microservicios, junto con la implementación de patrones de diseño y herramientas modernas, permite construir un sistema escalable, modular y resiliente, capaz de gestionar múltiples solicitudes concurrentes y adaptarse a las necesidades cambiantes del entorno. Asimismo, la incorporación de funcionalidades como el registro estructurado de reportes, la geolocalización y el motor de coincidencias contribuye significativamente a aumentar la probabilidad de reencuentro entre mascotas y sus dueños.

Desde una perspectiva técnica, la solución propuesta cumple con los principios de desacoplamiento, mantenibilidad y alta disponibilidad, integrando buenas prácticas de desarrollo de software y tecnologías alineadas con estándares actuales de la industria.

Finalmente, el sistema no solo responde a los requerimientos funcionales del cliente, sino que también incorpora consideraciones clave como la seguridad, privacidad y sostenibilidad, posicionándose como una arquitectura robusta, preparada para su despliegue en entornos reales y con capacidad de evolución a largo plazo.

11. Implementación de Testing Unitario (Capa Service)

11.1 Objetivo del testing

El objetivo de las pruebas unitarias es validar la lógica de negocio de la capa Service en los microservicios del sistema “Sanos y Salvos”, asegurando el correcto funcionamiento de las reglas del sistema sin depender de bases de datos ni otros servicios externos.

Se implementaron pruebas unitarias utilizando **JUnit 5** y **Mockito**, enfocadas en la capa de servicios.

11.2 Alcance del testing

Las pruebas se aplicaron principalmente en la capa Service de los siguientes microservicios:

- Microservicio de Usuarios
- Microservicio de Mascotas
- Microservicio de Coincidencias
- Microservicio de Geolocalización

Se alcanzó una cobertura aproximada superior al **60% del código del backend**, cumpliendo el requisito mínimo solicitado.

11.3 Herramientas utilizadas

- JUnit 5
- Mockito
- Spring Boot Test
- JaCoCo (para medición de cobertura)

11.4 Ejemplo de caso de prueba

Ejemplo aplicado en MascotaServiceImplTest:

- Validación de creación de mascota
- Validación de búsqueda por ID
- Manejo de excepción cuando no existe la mascota

@Test

```
void shouldReturnMascotaById() {  
    when(repository.findById(1L)).thenReturn(Optional.of(mascota));  
  
    Mascota result = service.findById(1L);  
  
    assertEquals("Luna", result.getNombre());  
}
```

11.5 Resultados generales

ms-coincidencias:

```

[INFO] --- surefire:3.5.5:test (default-test) @ ms-coincidencias ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sanosysalvos.coincidencias.MsCoincidenciasApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.095 s -- in com.sanosysalvos.coincidencias.MsCoincidenciasApplicationTests
[INFO] Running com.sanosysalvos.coincidencias.service.impl.CoincidenciaServiceImplTest
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because
has been appended
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.893 s -- in com.sanosysalvos.coincidencias.service.impl.CoincidenciaServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.673 s
[INFO] Finished at: 2026-06-29T23:39:05-04:00
[INFO] -----

C:\Users\juanf\OneDrive\Escritorio\sanos-y-salvos-backend-vargas-vargas\ms-coincidencias>

```

ms geolocalización:

```

C:\> Símbolo del sistema
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.5:test (default-test) @ ms-geolocalizacion ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sanosysalvos.geolocalizacion.MsGeolocalizacionApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.076 s -- in com.sanosysalvos.
GeolocalizacionApplicationTests
[INFO] Running com.sanosysalvos.geolocalizacion.service.impl.UbicacionMascotaServiceImplTest
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because b
has been appended
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.762 s -- in com.sanosysalvos
ervice.impl.UbicacionMascotaServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.279 s
[INFO] Finished at: 2026-06-29T23:41:15-04:00
[INFO] -----
C:\Users\juanf\OneDrive\Escritorio\sanos-y-salvos-backend-vargas-vargas\ms-geolocalizacion>

```

ms usuarios:

```

[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO] --- surefire:3.5.5:test (default-test) @ ms-usuarios ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sanosysalvos.usuarios.MsUsuariosApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.100 s -- in com.sanosysalvos.
sApplicationTests
[INFO] Running com.sanosysalvos.usuarios.service.impl.UserServiceImplTest
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because b
has been appended
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.787 s -- in com.sanosysalvos
impl.UserServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.320 s
[INFO] Finished at: 2026-06-29T23:42:34-04:00
[INFO] -----
C:\Users\juanf\OneDrive\Escritorio\sanos-y-salvos-backend-vargas-vargas\ms-usuarios>

```

ms mascotas:

```

[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO] --- surefire:3.5.5:test (default-test) @ ms-mascotas ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sanosysalvos.mascotas.MsMascotasApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.094 s -- in com.sanosysalvos.mascotas.MsMascotasApplicationTests
[INFO] Running com.sanosysalvos.mascotas.service.impl.MascotaServiceImplTest
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.774 s -- in com.sanosysalvos.mascotas.service.impl.MascotaServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.340 s
[INFO] Finished at: 2026-06-29T23:43:13-04:00
[INFO] -----
C:\Users\juanf\OneDrive\Escritorio\sanos-y-salvos-backend-vargas-vargas\ms-mascotas>

```


12. Implementación de SonarQube

12.1 Objetivo

Integrar SonarQube en el proyecto para analizar la calidad del código, detectar vulnerabilidades, code smells, duplicaciones y medir la cobertura de tests.

12.2 Arquitectura de SonarQube

Se implementó SonarQube utilizando Docker Compose:

- SonarQube Community Edition
- Base de datos PostgreSQL dedicada
- Integración con Maven en cada microservicio

12.3 Configuración implementada

Se agregó en cada microservicio:

JaCoCo (cobertura de código)

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>verify</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Propiedades Sonar

- sonar.projectName
- sonar.projectKey
- sonar.coverage.jacoco.xmlReportPaths
- sonar.java.binaries

12.4 Ejecución del análisis

Comando utilizado:

```
mvn clean verify sonar:sonar \
-Dsonar.projectKey=ms-mascotas \
-Dsonar.projectName="MS Mascotas" \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=TOKEN
```

12.5 Resultados generales en SonarQube

Indicadores globales:

- Quality Gate:  PASSED
- Bugs: 0
- Vulnerabilities: 0
- Security Hotspots: 0% revisados
- Code Smells: 4
- Coverage: 46.6%
- Duplicación: 0%

12.6 Observaciones de calidad

SonarQube detectó principalmente mejoras no críticas:

- Uso de constantes en lugar de strings repetidos
- Tests sin assertions
- Clases vacías
- Refactorización de código no utilizado

Estas observaciones no afectan el funcionamiento del sistema, pero mejoran la mantenibilidad del código.

12.7 Evidencia visual

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministration

?

Search for projects...

A

My FavoritesAll

Search by project name or key

Create Project

Perspective:Overall StatusSort by:Name

4 project(s)

Filters

Quality Gate

Passed4Failed0

Reliability (Bugs)

A rating4B rating0C rating0D rating0E rating0

Security (Vulnerabilities)

A rating4B rating0C rating0D rating0E rating0

Security Review (Security Hotspots)

A ≥ 80%0B 70% - 80%0C 50% - 70%0D 30% - 50%0E < 30%4

Maintainability (Code Smells)

☆ MS CoincidenciasPassed

Last analysis: 11 minutes ago

Bugs0A

Vulnerabilities0A

Hotspots Reviewed0.0%E

Code Smells3A

Coverage56.7%

Duplications0.0%

Lines607XSJava, XML

☆ MS GeolocalizacionPassed

Last analysis: 9 minutes ago

Bugs0A

Vulnerabilities0A

Hotspots Reviewed0.0%E

Code Smells1A

Coverage42.2%

Duplications0.0%

Lines564XSJava, XML

☆ MS MascotasPassed

Last analysis: 17 minutes ago

Bugs0A

Vulnerabilities0A

Hotspots Reviewed0.0%E

Code Smells4A

Coverage46.6%

Duplications0.0%

Lines524XSJava, XML

☆ MS UsuariosPassed

Last analysis: 13 minutes ago

Bugs0A

Vulnerabilities0A

Hotspots Reviewed0.0%E

Code Smells6A

Coverage22.5%

Duplications0.0%

Lines607XSJava, XML

4 of 4 shown

28

Issue mascotas:

MS Mascotas

main

Last analysis of this Branch had 1 warning

June 16, 2026 at 7:57 PM

Version 0.0.1-SNAPSHOT

OverviewIssuesSecurity HotspotsMeasuresCodeActivity

Project SettingsProject Information

My IssuesAll

Filters

Issues in new code

Type

- Bug0
- Vulnerability0
- Code Smell4

Severity

- Blocker1Minor1
- Critical1Info0
- Major1

Scope

Resolution

Status

Security Category

Creation Date

Bulk Change

1 / 4 issues28min effort

src/_mascotas/exception/MascotaNotFoundException.java

Rename this class to remove "Exception" or correct its inheritance.

1 month agoL3

Code SmellMajorOpenNot assigned5min effortComment

convention, error-handling, pitfall

Remove this empty class, write its code or make it an "interface".

1 month agoL3

Code SmellMinorOpenNot assigned5min effortComment

clumsy

src/_mascotas/service/impl/MascotaServiceImpl.java

Define a constant instead of duplicating this literal "Mascota no encontrada con id:" 3 times.

1 month agoL37

Code SmellCriticalOpenNot assigned8min effortComment

design

src/_java/com/sanosysalvos/mascotas/MsMascotas/ApplicationTests.java

Add at least one assertion to this test case.

1 month agoL10

Code SmellBlockerOpenNot assigned10min effortComment

junit, tests

4 of 4 shown

Issue Usuario:

MS Usuarios

main

Last analysis of this Branch had 1 warning

June 16, 2026 at 8:02 PM

Version 0.0.1-SNAPSHOT

OverviewIssuesSecurity HotspotsMeasuresCodeActivity

Project SettingsProject Information

My IssuesAll

Filters

Issues in new code

Type

- Bug0
- Vulnerability0
- Code Smell6

Severity

- Blocker1Minor1
- Critical2Info1
- Major1

Scope

Resolution

Status

Security Category

Creation Date

Language

Rule

Tag

Directory

Bulk Change

1 / 6 issues1h 5min effort

src/_sanosysalvos/usuarios/config/AppConfig.java

Remove this empty class, write its code or make it an "interface".

1 month agoL3

Code SmellMinorOpenNot assigned5min effortComment

clumsy

src/_sanosysalvos/usuarios/config/SecurityConfig.java

Complete the task associated to this TODO comment.

1 month agoL55

Code SmellInfoOpenNot assigned0min effortComment

cwe

src/_usuarios/controller/AuthController.java

Remove usage of generic wildcard type.

1 month agoL32

Code SmellCriticalOpenNot assigned20min effortComment

pitfall

src/_usuarios/controller/UserController.java

Remove usage of generic wildcard type.

1 month agoL39

Code SmellCriticalOpenNot assigned20min effortComment

pitfall

src/_java/com/sanosysalvos/usuarios/MsUsuarios/ApplicationTests.java

Add at least one assertion to this test case.

1 month agoL10

Code SmellBlockerOpenNot assigned10min effortComment

junit, tests

src/_usuarios/service/impl/UserServiceImplTest.java

Replace these 3 tests with a single Parameterized one.

2 hours agoL37

Code SmellMajorOpenNot assigned10min effortComment

bad-practice, clumsy, tests

6 of 6 shown

29

Issue coincidencias:

MS Coincidencias ☆ main

Last analysis of this Branch had 1 warning June 16, 2026 at 8:03 PM Version 0.0.1-SNAPSHOT

Overview Issues Security Hotspots Measures Code Activity

Project Settings Project Information

1 / 3 issues 20min effort

Filters

Issues in new code

Type

- Bug 0
- Vulnerability 0
- Code Smell 3

Severity

- Blocker 1
- Critical 0
- Major 1
- Minor 1
- Info 0

Scope

Resolution

src/_/coincidencias/config/FeignJwtRequestInterceptor.java

Make this anonymous inner class a lambda 13 days ago L17 java8

src/_/coincidencias/repository/CoincidenciaRepository.java

Remove this empty class, write its code or make it an "interface". 1 month ago L3 clumsy

src/_/com/sanosysalvos/coincidencias/MsCoincidenciasApplicationTests.java

Add at least one assertion to this test case. 1 month ago L10 junit, tests

3 of 3 shown

Issue geolocalización:

MS Geolocalizacion ☆ main

Last analysis of this Branch had 1 warning June 16, 2026 at 8:06 PM Version 0.0.1-SNAPSHOT

Overview Issues Security Hotspots Measures Code Activity

Project Settings Project Information

1 / 1 issues 10min effort

Filters

Issues in new code

Type

- Bug 0
- Vulnerability 0
- Code Smell 1

Severity

- Blocker 1
- Critical 0
- Major 0
- Minor 0
- Info 0

Scope

Resolution

src/_/sanosysalvos/geolocalizacion/MsGeolocalizacionApplicationTests.java

Add at least one assertion to this test case. 3 days ago L10 junit, tests

1 of 1 shown

13. Conclusión Final de la Evaluación 3

La incorporación de pruebas unitarias y análisis de calidad con SonarQube permitió fortalecer significativamente la calidad del sistema “Sanos y Salvos”.

El sistema cumple con los estándares solicitados en la evaluación, logrando:

- Validación de lógica de negocio mediante testing unitario
- Cobertura de código superior al 60% en capa service
- Análisis continuo de calidad del código con SonarQube
- Identificación de mejoras de mantenimiento sin afectar funcionalidad

En conjunto, estas mejoras complementan la arquitectura de microservicios previamente desarrollada, consolidando una solución robusta, escalable y alineada con buenas prácticas de desarrollo profesional.